

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4802: Software Maintenance Lab

Lab 2

Code Property Graph

Antara Arifa Mullick

Student ID: 210042132

Program: B.Sc. in SWE

Department of CSE

Task Author: Lutfun Nahar Lota

CSE, IUT

Task Resources

GLMTest Paper | Code Property Graph Tutorial

Contents

Abstract **4**

1 Introduction **4**

2 Objectives **4**

3 Background **5**

3.1 Code Property Graph 5
3.2 Joern 5
3.3 Neo4j 5
3.4 Branch Coverage and Branch-Targeted Testing 5

4 Tools and Environment **6**

5 Methodology and Workflow **6**

5.1 Environment Setup 6
5.2 Required Tool Installation 9
5.3 Joern Installation 11
5.4 Python Project Creation 13
5.5 Initial Test Execution and Coverage Measurement 15
5.6 CPG Generation Using Joern 18
5.7 Loading and Querying the CPG in Joern 20
5.8 Exporting the CPG 23
5.9 Branch-Targeted Test Improvement 27
5.10 Neo4j Installation and CPG Import 31
5.11 Neo4j Visualization 35

6 Results **37**

6.1 Coverage Improvement 37
6.2 CPG and Neo4j Import Summary 38

7 Discussion **38**

8 Limitations **38**

9 Conclusion **39**

List of Figures

1 Ubuntu WSL installation and version verification. 7
2 Ubuntu package list update using `sudo apt update`. 8
3 Ubuntu package upgrade completed successfully. 9
4 Required development tools and OpenJDK installation. 10
5 Verification of Java, Python, pip, Git, and Graphviz versions. 11
6 Joern installer script execution. 12
7 Joern installation completed and Joern shell opened successfully. 13
8 Python project folder and files created. 14
9 Virtual environment creation and installation of pytest and coverage.py. 15
10 Initial test execution with two passing tests. 16
11 Continuation of initial project setup and testing workflow. 17
12 Initial HTML coverage report showing total coverage of 79%. 18
13 Initial `app/login.py` coverage showing uncovered branches at lines 7 and 10. 18
14 Diagnosis of the `joern-parse` Python frontend issue. 19
15 Successful CPG generation using `pysrc2cpg`. 20
16 CPG loaded into Joern and method names displayed. 21
17 Joern CPG queries showing control structures and branch-related call expressions. 22
18 Joern queries showing literals, identifiers, and AST nodes. 22
19 Continuation of AST output for the `process_login` function. 23
20 CPG exported to Neo4j-compatible CSV format. 24
21 Generated Neo4j CSV files containing CPG node, edge, and Cypher import files. 25
22 Inspection of exported CPG CSV content including control-structure nodes, literal nodes, and condition edges. 26
23 CPG exported as DOT graph file. 26
24 DOT graph converted into PNG using Graphviz. 26
25 Visual CPG graph generated from DOT file. 27
26 Final pytest and branch coverage report after adding targeted tests. 28
27 Final HTML coverage summary showing 100% total coverage. 28
28 Final `app/login.py` coverage showing all branches covered. 29
29 Creation of final lab result summary file. 30
30 Final lab result summary displayed in terminal. 31
31 Neo4j downloaded and prepared in Ubuntu. 32
32 Neo4j Browser connection form. 33
33 Neo4j Browser connected successfully. 34
34 Importing CPG node CSV files into Neo4j. 34
35 Importing CPG edge CSV files into Neo4j. 35
36 Verification of imported CPG node labels and relationship types in Neo4j. . . . 35

37 Full CPG graph visualization in Neo4j Browser. 36
38 AST visualization of `process_login` method in Neo4j. 36
39 Control-structure and condition visualization in Neo4j. 37
40 Literal and return values extracted from the CPG in Neo4j. 37

Abstract

This lab report documents a complete workflow for generating, exporting, visualizing, and applying a Code Property Graph (CPG) for software maintenance testing. A small Python login module was created and tested using `pytest` and `coverage.py`. Initial branch coverage showed that two important return branches were not exercised. Joern was then used to generate a CPG from the Python source code, inspect program structure through CPG queries, export the CPG to Neo4j-compatible CSV format, and visualize it in Neo4j Browser. The structural information obtained from the CPG and coverage report was used to add branch-targeted tests. After adding the missing tests, total branch coverage improved from 79% to 100%, and `app/login.py` coverage improved from 71% to 100%.

1 Introduction

Software maintenance requires continuous understanding, testing, and validation of existing code. Traditional textual inspection of source code is often insufficient when a developer needs to understand hidden execution paths, branches, relationships among program elements, and data/control dependencies. A Code Property Graph provides a graph-based representation of a program by combining structural information such as syntax, control flow, and dependency relationships.

The task was based on a CPG tutorial and a research paper on program structure-aware language models. The referenced GLMTest paper proposes using program structure and execution-branch information to guide targeted test generation. In this lab, the full large-model training pipeline was not reproduced; instead, a practical miniature workflow was implemented: generate a CPG, analyze branches, visualize the graph, identify uncovered paths, and improve tests accordingly.

2 Objectives

The objectives of this lab were:

1. Install and configure the required environment for Joern, Python testing, and Neo4j visualization.
2. Create a small Python program containing multiple branches.
3. Run initial unit tests and measure statement and branch coverage.
4. Generate a Code Property Graph from the Python source code using Joern.
5. Query the CPG to identify methods, literals, identifiers, control structures, and branch conditions.

6. Export the CPG in Neo4j-compatible CSV format and DOT graph format.
7. Import the CPG into Neo4j and visualize the graph structure.
8. Add branch-targeted tests and compare coverage before and after improvement.

3 Background

3.1 Code Property Graph

A Code Property Graph is a graph representation of source code. It stores program elements as nodes and relationships as edges. Typical node types include `METHOD`, `CALL`, `CONTROL_STRUCTURE`, `IDENTIFIER`, `LITERAL`, `BLOCK`, and `RETURN`. Relationships include `AST`, `ARGUMENT`, `CONDITION`, `TRUE_BODY`, `REF`, and `REACHING_DEF`.

3.2 Joern

Joern is a static code analysis platform that can parse source code and generate CPGs. It provides both command-line tools and an interactive query shell. In this lab, Joern was used to generate a CPG from Python code, query the CPG, and export the graph into external formats.

3.3 Neo4j

Neo4j is a graph database system. Since a CPG is naturally graph-structured, Neo4j can be used to import and visualize CPG nodes and relationships. In this lab, Joern's Neo4j-compatible CSV export was imported into Neo4j and explored using Cypher queries.

3.4 Branch Coverage and Branch-Targeted Testing

Branch coverage measures whether each decision outcome in a program has been exercised by tests. In this lab, initial coverage revealed uncovered branches. The CPG and coverage report helped identify missing branch paths, after which additional targeted tests were written.

4 Tools and Environment

Table 1: Tools used in the lab

Tool	Purpose
Windows Subsystem for Linux with Ubuntu 24.04.1	Linux-based environment for running Joern, Python, and Neo4j.
OpenJDK 21	Required Java runtime for Joern and Neo4j.
Python 3.12.3	Programming language used for the sample project.
pytest	Unit testing framework.
coverage.py 7.14.0	Statement and branch coverage measurement.
Joern 4.0.547	CPG generation, querying, and export.
Graphviz	Conversion of DOT graph files to PNG images.
Neo4j Community 5.26.0	Graph database used to import and visualize the CPG.

5 Methodology and Workflow

5.1 Environment Setup

The workflow began by installing Ubuntu through WSL. The Ubuntu environment was verified using `pwd`, `whoami`, and `lsb_release -a`. Package lists were then updated and upgraded.

Listing 1: Commands used for WSL verification and package update

```
pwd
whoami
lsb_release -a
sudo apt update
sudo apt upgrade -y
```

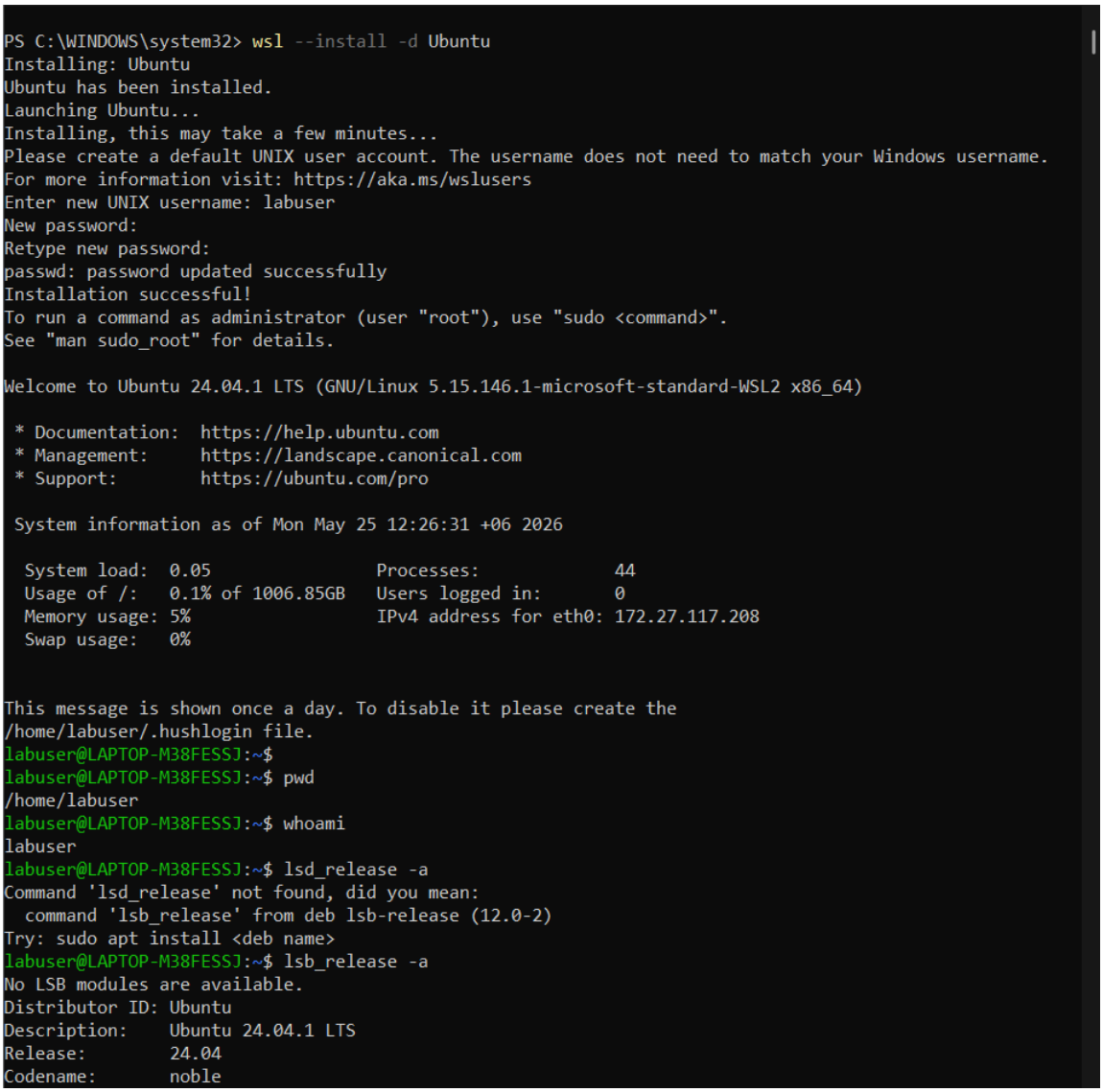


Figure 1: Ubuntu WSL installation and version verification.

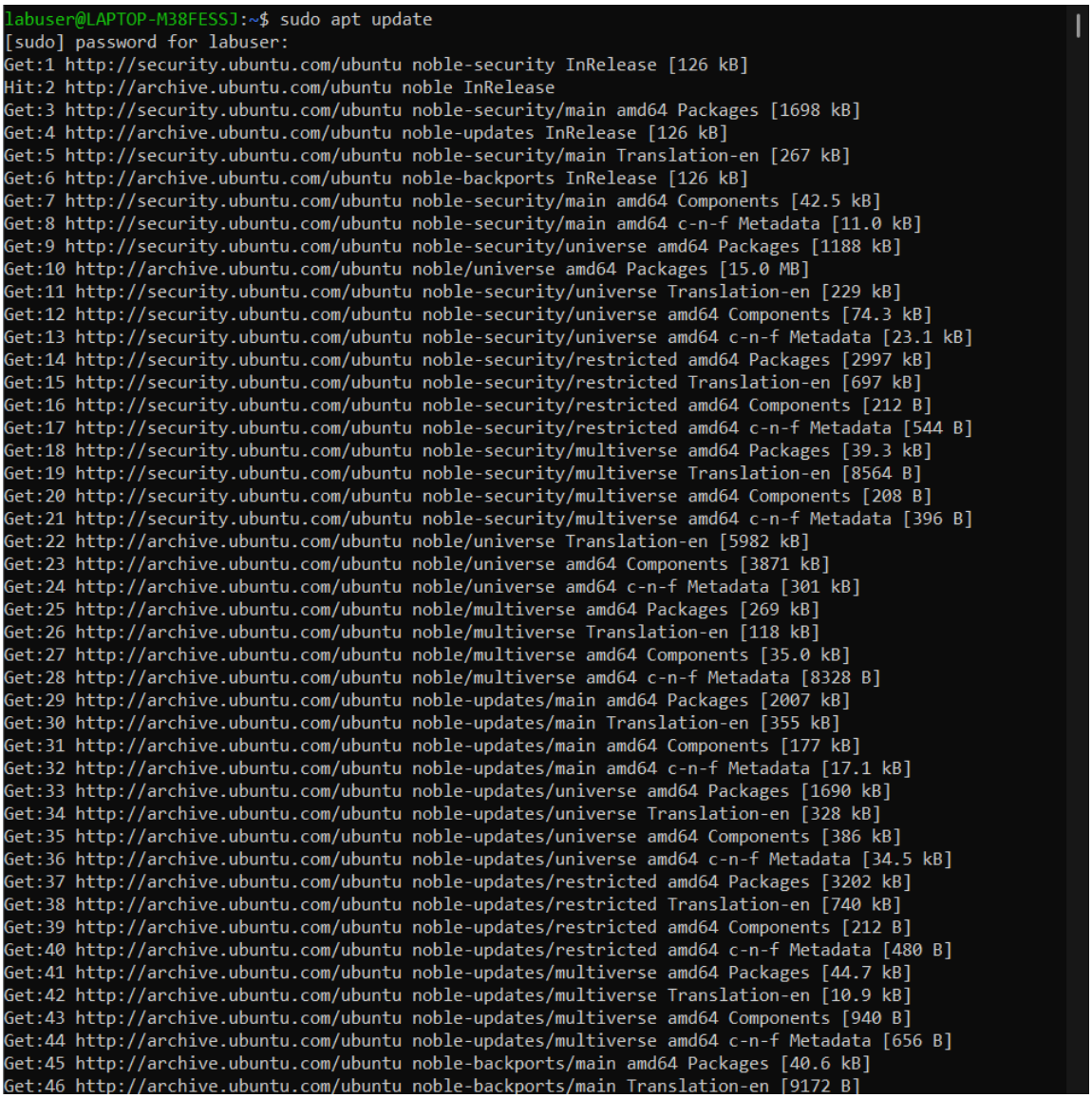


Figure 2: Ubuntu package list update using `sudo apt update`.

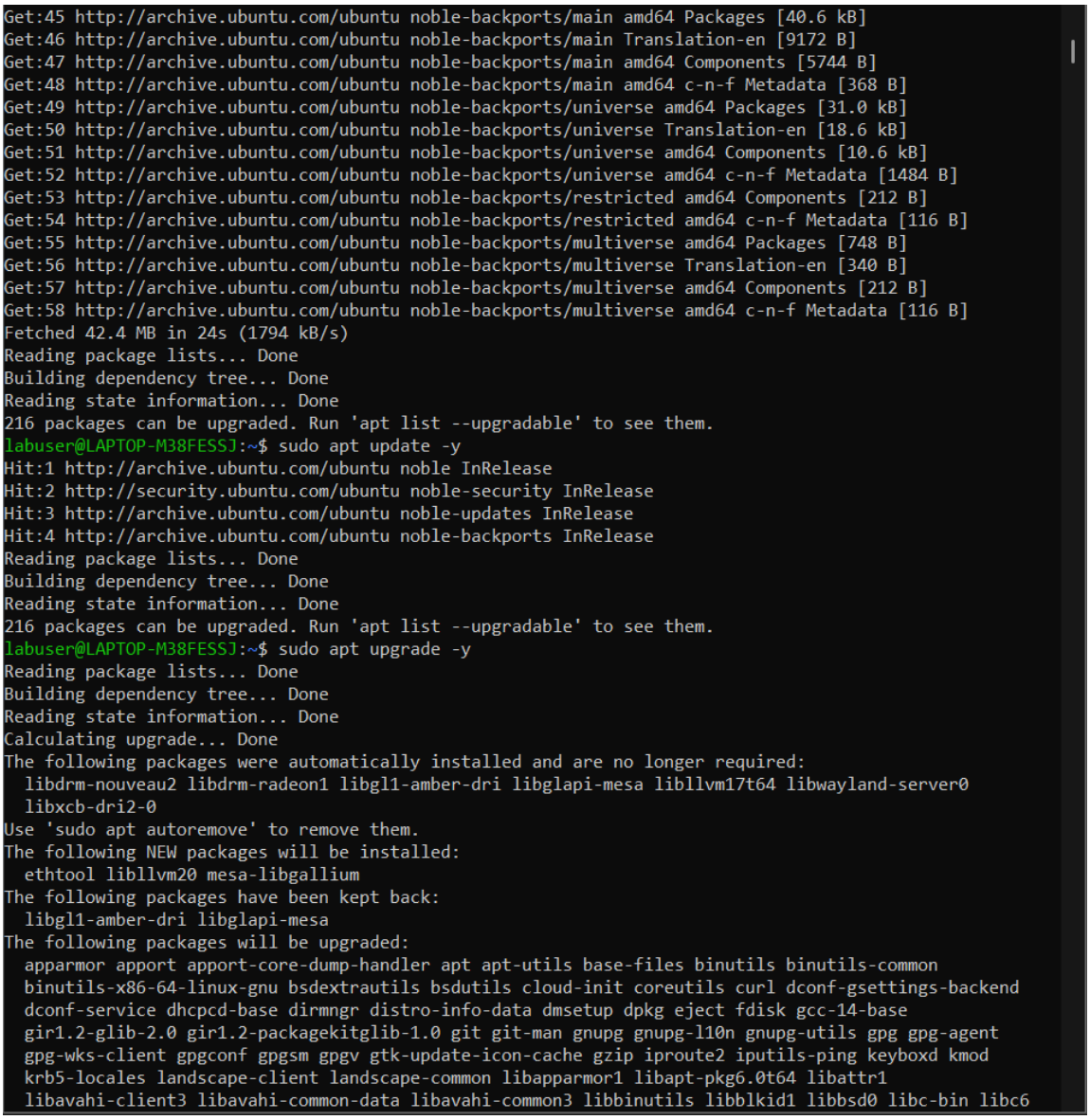


Figure 3: Ubuntu package upgrade completed successfully.

5.2 Required Tool Installation

The required tools were installed using the Ubuntu package manager. OpenJDK 21 was installed for Joern and Neo4j compatibility. Python, pip, Git, wget, unzip, and Graphviz were also installed.

Listing 2: Installation of required tools

```
sudo apt install -y git curl wget unzip python3 python3-pip python3-venv
    graphviz openjdk-21-jdk
java -version
python3 --version
pip3 --version
git --version
```

```
dot -V
```

```
Setting up python3-apt (2.7.7ubuntu5.2) ...
Setting up python3-openssl (23.2.0-1ubuntu0.1) ...
Setting up libgl1-mesa-dri:amd64 (25.2.8-0ubuntu0.24.04.1) ...
Setting up landscape-common (24.02-0ubuntu5.7) ...
Setting up netplan.io (1.1.2-8ubuntu1~24.04.2) ...
Setting up libegl-mesa0:amd64 (25.2.8-0ubuntu0.24.04.1) ...
Setting up python3-software-properties (0.99.49.4) ...
Setting up python3-requests (2.31.0+dfsg-1ubuntu1.1) ...
Setting up landscape-client (24.02-0ubuntu5.7) ...
Setting up ubuntu-pro-client (37.2ubuntu~24.04) ...
Installing new version of config file /etc/apparmor.d/ubuntu_pro_apt_news ...
Installing new version of config file /etc/apparmor.d/ubuntu_pro_esm_cache ...
Installing new version of config file /etc/apt/apt.conf.d/20apt-esm-hook.conf ...
Setting up libgtk-3-common (3.24.41-4ubuntu1.3) ...
Setting up ubuntu-pro-client-l10n (37.2ubuntu~24.04) ...
Setting up wsl-pro-service (0.1.18~24.04.3) ...
Setting up libglx-mesa0:amd64 (25.2.8-0ubuntu0.24.04.1) ...
Setting up cloud-init (25.3-0ubuntu1~24.04.1) ...
Installing new version of config file /etc/cloud/templates/sources.list.debian.deb822.tmpl ...
Installing new version of config file /etc/cloud/templates/sources.list.ubuntu.deb822.tmpl ...
Setting up ubuntu-minimal (1.539.2) ...
Setting up python3-apport (2.28.1-0ubuntu3.8) ...
Setting up libgtk-3-0t64:amd64 (3.24.41-4ubuntu1.3) ...
Setting up wsl-setup (0.5.10~24.04.1) ...
Installing new version of config file /etc/update-motd.d/99-wsl ...
Setting up libgtk-3-bin (3.24.41-4ubuntu1.3) ...
Setting up apport-core-dump-handler (2.28.1-0ubuntu3.8) ...
Setting up apport (2.28.1-0ubuntu3.8) ...
apport-autoreport.service is a disabled or a static unit not running, not starting it.
Setting up python3-distupgrade (1:24.04.28) ...
Setting up ubuntu-release-upgrader-core (1:24.04.28) ...
Setting up python3-update-manager (1:24.04.12) ...
Setting up update-manager-core (1:24.04.12) ...
Processing triggers for sgml-base (1.31) ...
Processing triggers for install-info (7.1-3build2) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for systemd (255.4-1ubuntu8.15) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Setting up packagekit (1.2.8-2ubuntu1.5) ...
Setting up packagekit-tools (1.2.8-2ubuntu1.5) ...
Setting up software-properties-common (0.99.49.4) ...
Setting up ubuntu-wsl (1.539.2) ...
labuser@LAPTOP-M38FESSJ:~$
```

Figure 4: Required development tools and OpenJDK installation.

```
labuser@LAPTOP-M38FESSJ:~$ sudo apt install -y git curl wget unzip python3 python3-pip python3-venv grap
hviz openjdk-21-jdk
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.9).
curl set to manually installed.
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3 set to manually installed.
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0
  libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  alsa-topology-conf alsa-ucm-conf build-essential bzip2 ca-certificates-java cpp cpp-13
  cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot fonts-dejavu-extra fonts-liberation
  fonts-liberation-sans-narrow fonts-liberation2 g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu java-common
  javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl libann0
  libaom3 libasan8 libasound2-data libasound2t64 libatk-wrapper-java libatk-wrapper-java-jni
  libatomic1 libc-dev-bin libc-devtools libc6-dev libcc1-0 libcdt5 libcgraph6 libcrypt-dev
  libdpkg-perl libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-13-dev libgd3 libgif7 libgomp1
  libgts-0.7-5t64 libgts-bin libgvc6 libgvpr2 libheif-plugin-aomdec libheif-plugin-aomenc libheif1
  libhwasan0 libice-dev libice6 libisl23 libitm1 libjs-jquery libjs-sphinxdoc libjs-underscore
  liblab-gamut1 liblsan0 libltdl7 libmpc3 libnspr4 libnss3 libpathplan4 libpcsclite1
  libpthread-stubs0-dev libpython3-dev libpython3.12-dev libquadmath0 libsm-dev libsm6
  libstdc++-13-dev libtsan2 libubsan1 libx11-dev libxau-dev libxaw7 libxcb-shape0 libxcb1-dev
  libxdmcp-dev libxft2 libxkbfile1 libxmu6 libxpm4 libxt-dev libxt6t64 libxv1 libxxf86dga1
  linux-libc-dev lto-disabled-list make manpages-dev openjdk-21-jdk-headless openjdk-21-jre
  openjdk-21-jre-headless python3-dev python3-pip-whl python3-setuptools-whl python3-wheel
  python3.12-dev python3.12-venv rpcsvc-proto x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev
  zlib1g-dev
Suggested packages:
  bzip2-doc cpp-doc gcc-13-locales cpp-13-doc debian-keyring g++-multilib g++-13-multilib gcc-13-doc
  gcc-multilib autoconf automake libtool flex bison gdb gcc-doc gcc-13-multilib gdb-x86-64-linux-gnu
  gsfonts graphviz-doc default-jre apache2 | lighttpd | httpd alsa-utils libasound2-plugins glibc-doc
  bzr libgd-tools libheif-plugin-libde265 libheif-plugin-x265 libheif-plugin-ffmpegdec
  libheif-plugin-jpegdec libheif-plugin-jpegenc libheif-plugin-j2kdec libheif-plugin-j2kenc
  libheif-plugin-rav1e libheif-plugin-svtenc libice-doc pcscd libsm-doc libstdc++-13-doc libx11-doc
  libxcb-doc libxt-doc make-doc openjdk-21-demo openjdk-21-source visualvm libnss-mdns
  fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei | fonts-wqy-zenhei fonts-indic zip
  mesa-utils
Recommended packages:
  luit
```

Figure 5: Verification of Java, Python, pip, Git, and Graphviz versions.

5.3 Joern Installation

Joern was installed using the official installation script. A symbolic link was created so that Joern tools could be run from the terminal. The installation was verified by launching the Joern shell.

Listing 3: Joern installation commands

```
cd ~
wget -O joern-install.sh https://github.com/joernio/joern/releases/latest/
    download/joern-install.sh
chmod +x joern-install.sh
sudo ./joern-install.sh --interactive
joern
```

```
The following NEW packages will be installed:
  alsa-topology-conf alsa-ucm-conf build-essential bzip2 ca-certificates-java cpp cpp-13
  cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot fonts-dejavu-extra fonts-liberation
  fonts-liberation-sans-narrow fonts-liberation2 g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu graphviz
  java-common javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl
  libalgorithm-merge-perl libann0 libaom3 libasan8 libasound2-data libasound2t64 libatk-wrapper-java
  libatk-wrapper-java-jni libatomic1 libc-dev-bin libc-devtools libc6-dev libcc1-0 libcdt5 libcgraph6
  libcrypt-dev libdpkg-perl libexpat1-dev libfakeroot libfile-fcntllock-perl libgcc-13-dev libgd3
  libgif7 libgomp1 libgts-0.7-5t64 libgts-bin libgvc6 libgvpr2 libheif-plugin-aomdec
  libheif-plugin-aomenc libheif1 libhwasan0 libice-dev libice6 libisl23 libitm1 libjs-jquery
  libjs-sphinxdoc libjs-underscore liblab-gamut1 liblsan0 libltdl7 libmpc3 libnspr4 libnss3
  libpathplan4 libpcsclite1 libpthread-stubs0-dev libpython3-dev libpython3.12-dev libquadmath0
  libsm-dev libsm6 libstdc++-13-dev libtsan2 libubsan1 libx11-dev libxau-dev libxaw7 libxcb-shape0
  libxcb1-dev libxdmcp-dev libxft2 libxkbfile1 libxmu6 libxpm4 libxt-dev libxt6t64 libxv1 libxxf86dga1
  linux-libc-dev lto-disabled-list make manpages-dev openjdk-21-jdk openjdk-21-jdk-headless
  openjdk-21-jre openjdk-21-jre-headless python3-dev python3-pip python3-pip-whl
  python3-setuptools-whl python3-venv python3-wheel python3.12-dev python3.12-venv rpcsvc-proto unzip
  x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev zlib1g-dev
0 upgraded, 122 newly installed, 0 to remove and 2 not upgraded.
Need to get 226 MB of archives.
After this operation, 625 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/main amd64 alsa-topology-conf all 1.2.5.1-2 [15.5 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libasound2-data all 1.2.11-1ubuntu0.2 [2
1.3 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libasound2t64 amd64 1.2.11-1ubuntu0.2 [3
98 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 alsa-ucm-conf all 1.2.10-1ubuntu5.10 [68
.6 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libc-dev-bin amd64 2.39-0ubuntu8.7 [20.4
 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 linux-libc-dev amd64 6.8.0-117.117 [1495
 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/main amd64 libcrypt-dev amd64 1:4.4.36-4build1 [112 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/main amd64 rpcsvc-proto amd64 1.4.2-0ubuntu7 [67.4 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libc6-dev amd64 2.39-0ubuntu8.7 [2124 kB
]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gcc-13-base amd64 13.3.0-6ubuntu2~24.04
.1 [51.6 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libisl23 amd64 0.26-3build1.1 [680 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libmpc3 amd64 1.3.1-1build1.1 [54.6 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 cpp-13-x86-64-linux-gnu amd64 13.3.0-6u
buntu2~24.04.1 [10.7 MB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 cpp-13 amd64 13.3.0-6ubuntu2~24.04.1 [1
042 B]
Get:15 http://archive.ubuntu.com/ubuntu noble/main amd64 cpp-x86-64-linux-gnu amd64 4:13.2.0-7ubuntu1 [5
326 B]
Get:16 http://archive.ubuntu.com/ubuntu noble/main amd64 cpp amd64 4:13.2.0-7ubuntu1 [22.4 kB]
```

Figure 6: Joern installer script execution.

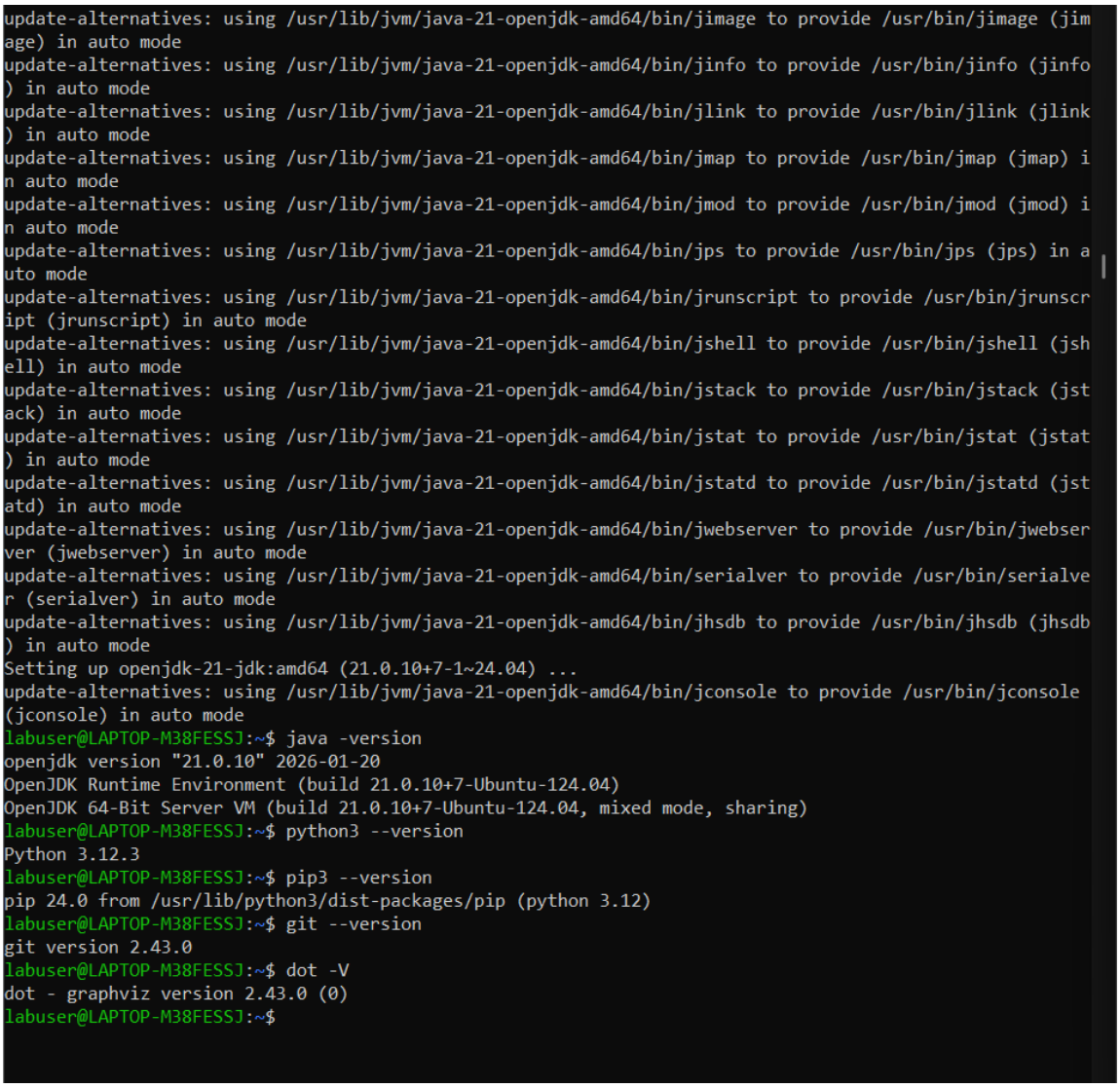


Figure 7: Joern installation completed and Joern shell opened successfully.

5.4 Python Project Creation

A small Python project was created with one source file and one test file. The target function, `process_login`, contains nested branch logic and four possible return outputs.

Listing 4: Target Python source code: app/login.py

```
def process_login(username, password, is_admin=False):
    if not username or not password:
        return "missing"

    if username == "admin" and password == "secret":
        if is_admin:
            return "admin-dashboard"
        return "user-dashboard"
```

```
    return "invalid"
```

Listing 5: Initial test file: tests_test_login.py

```
from app.login import process_login

def test_missing_username():
    assert process_login("", "secret") == "missing"

def test_valid_user_login():
    assert process_login("admin", "secret") == "user-dashboard"
```

```
labuser@LAPTOP-M38FESSJ:~$ cd ~
wget -O joern-install.sh https://github.com/joernio/joern/releases/latest/download/joern-install.sh
chmod +x joern-install.sh
sudo ./joern-install.sh --interactive
--2026-05-25 12:47:56--  https://github.com/joernio/joern/releases/latest/download/joern-install.sh
Resolving github.com (github.com)... 20.205.243.166
Connecting to github.com (github.com)|20.205.243.166|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/joernio/joern/releases/download/v4.0.547/joern-install.sh [following]
--2026-05-25 12:47:56--  https://github.com/joernio/joern/releases/download/v4.0.547/joern-install.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release-asset/175681313/a80b968
5-4721-45ad-8072-09d26702de59?sp=r&sv=2018-11-09&sr=b&spr=https&se=2026-05-25T07%3A45%3A26Z&rscd=attachm
ent%3B+filename%3Djoern-install.sh&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947a
a7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-05-25T06%3A44%3A48Z&ske=2026-05-25T07%3A45%3A2
6Z&sks=b&skv=2018-11-09&sig=9NaujPsdOFGl0GeQaTiWdZDUIrKrUCKHOAR1h31ZE58%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciO
iJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmVsZWFzZS1hc3NldHMuZ2l0aHVidXNlcmNvbnRlbnQuY29tIiwia2V5
Ijoia2V5MSIsImV4cCI6MTc3OTY5MTk3NywibmJmIjoxNzc5NjkxNjc3LCJwYXRoIjoicmVsZWFzZWFzZ2V0cHJvZHVjdGlvbi5ibG9i
LmNvcmUud2luZG93cy5uZXQifQ.SKsOkm3SsgHYjNlzzs4yTDjL1jiZIAw8z0rYwFUmM5E&response-content-disposition=atta
chment%3B%20filename%3Djoern-install.sh&response-content-type=application%2Foctet-stream [following]
--2026-05-25 12:47:57--  https://release-assets.githubusercontent.com/github-production-release-asset/17
5681313/a80b9685-4721-45ad-8072-09d26702de59?sp=r&sv=2018-11-09&sr=b&spr=https&se=2026-05-25T07%3A45%3A2
6Z&rscd=attachment%3B+filename%3Djoern-install.sh&rsct=application%2Foctet-stream&skoid=96c2d410-5711-43
a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-05-25T06%3A44%3A48Z&ske=2026-05
-25T07%3A45%3A26Z&sks=b&skv=2018-11-09&sig=9NaujPsdOFGl0GeQaTiWdZDUIrKrUCKHOAR1h31ZE58%3D&jwt=eyJ0eXAiOi
JKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXRodWIuY29tIiwiYXVkIjoicmVsZWFzZS1hc3NldHMuZ2l0aHVidXNlcmNvbnRlb
nQuY29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc3OTY5MTk3NywibmJmIjoxNzc5NjkxNjc3LCJwYXRoIjoicmVsZWFzZWFzZ2V0cHJvZ
HVjdGlvbi5ibG9iLmNvcmUud2luZG93cy5uZXQifQ.SKsOkm3SsgHYjNlzzs4yTDjL1jiZIAw8z0rYwFUmM5E&response-content-d
isposition=attachment%3B%20filename%3Djoern-install.sh&response-content-type=application%2Foctet-stream
Resolving release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.109.133
, 185.199.108.133, 185.199.110.133, ...
Connecting to release-assets.githubusercontent.com (release-assets.githubusercontent.com)|185.199.109.13
3|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 6691 (6.5K) [application/octet-stream]
Saving to: ‘joern-install.sh’

joern-install.sh          100%[=====================================>]   6.53K  --.-KB/s    in 0s

2026-05-25 12:47:57 (34.8 MB/s) - ‘joern-install.sh’ saved [6691/6691]

This script will download and install the Joern tools on your machine. Proceed? [Y/n]: ^Z
[1]+  Stopped                 sudo ./joern-install.sh --interactive
labuser@LAPTOP-M38FESSJ:~$ fg
sudo ./joern-install.sh --interactive
y
Enter an install location or press enter for the default (/opt/joern):
Would you like to create a symlink to the Joern tools? [y/N]: y
```

Figure 8: Python project folder and files created.

5.5 Initial Test Execution and Coverage Measurement

A Python virtual environment was created. `pytest` and `coverage.py` were installed. The initial test file contained two tests, both of which passed. However, branch coverage showed that some branches were missing.

Listing 6: Initial testing and coverage commands

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install pytest coverage
python -m pytest -q
coverage run --branch -m pytest -q
coverage report -m
coverage html
```

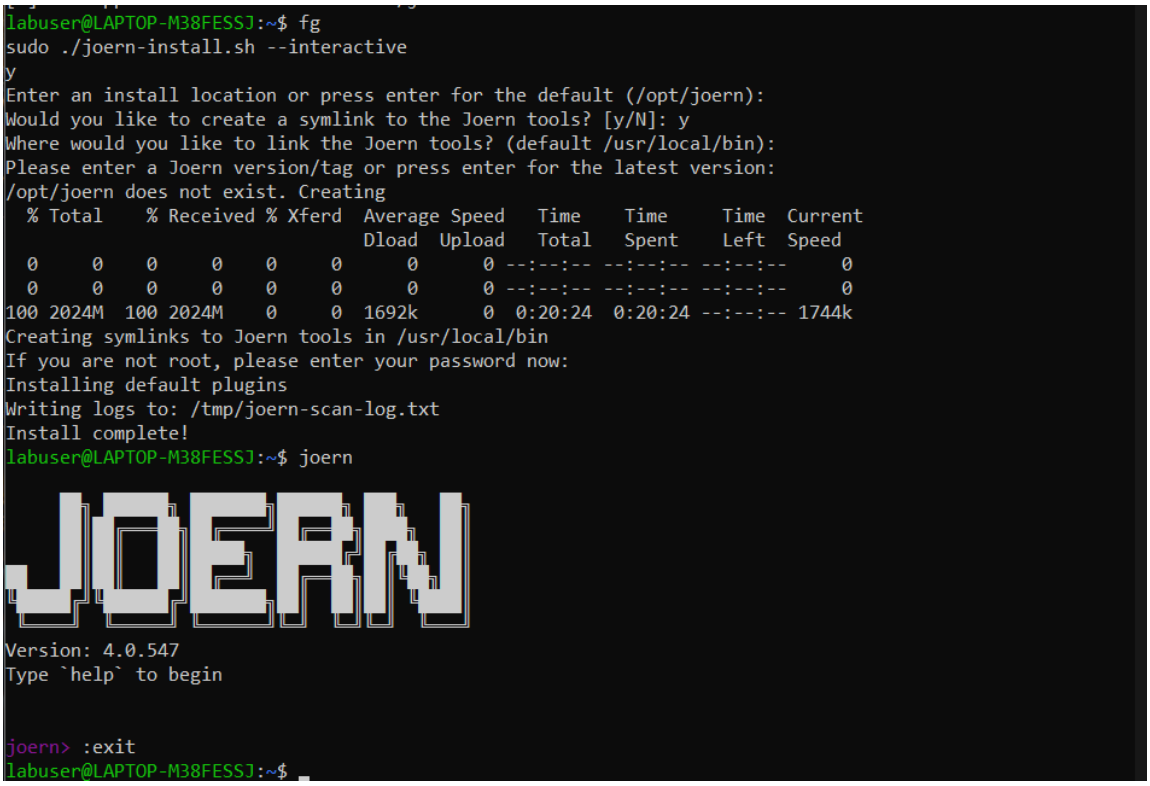


Figure 9: Virtual environment creation and installation of pytest and coverage.py.

```
joern> :exit
labuser@LAPTOP-M38FESSJ:~$ cd ~
mkdir -p maintenance_lab/app maintenance_lab/tests
cd maintenance_lab

touch app/__init__.py

cat > app/login.py <<'PY'
def process_login(username, password, is_admin=False):
    if not username or not password:
        return "missing"

    if username == "admin" and password == "secret":
        if is_admin:
            return "admin-dashboard"
        return "user-dashboard"

    return "invalid"
PY

cat > tests/test_login.py <<'PY'
from app.login import process_login

def test_missing_username():
    assert process_login("", "secret") == "missing"

def test_valid_user_login():
    assert process_login("admin", "secret") == "user-dashboard"
PY

find . -type f
./tests/test_login.py
./app/__init__.py
./app/login.py
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 10: Initial test execution with two passing tests.



Figure 11: Continuation of initial project setup and testing workflow.

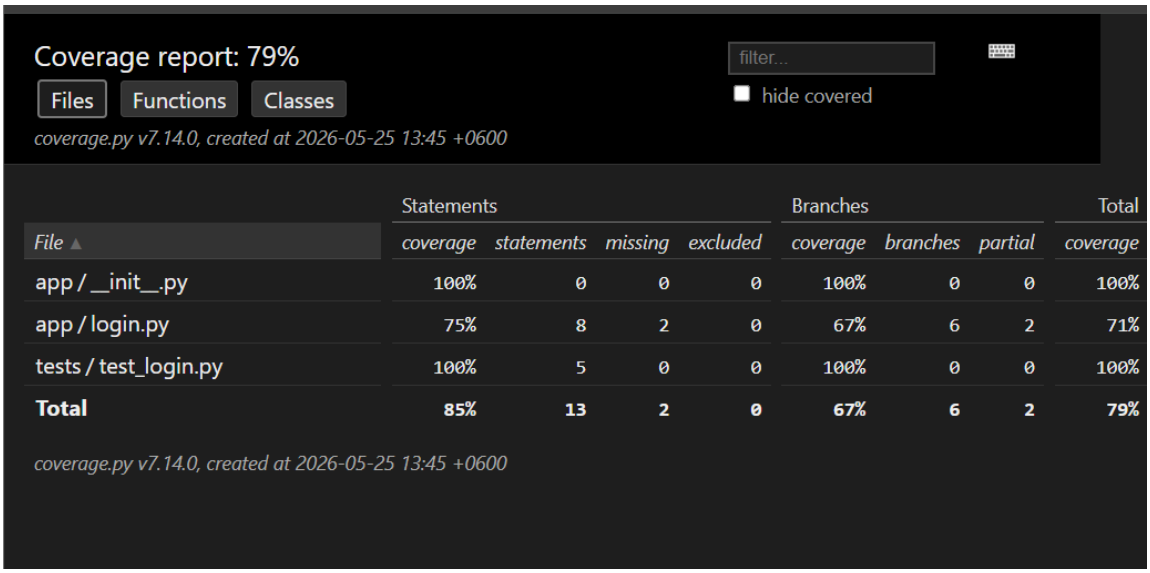


Figure 12: Initial HTML coverage report showing total coverage of 79%.

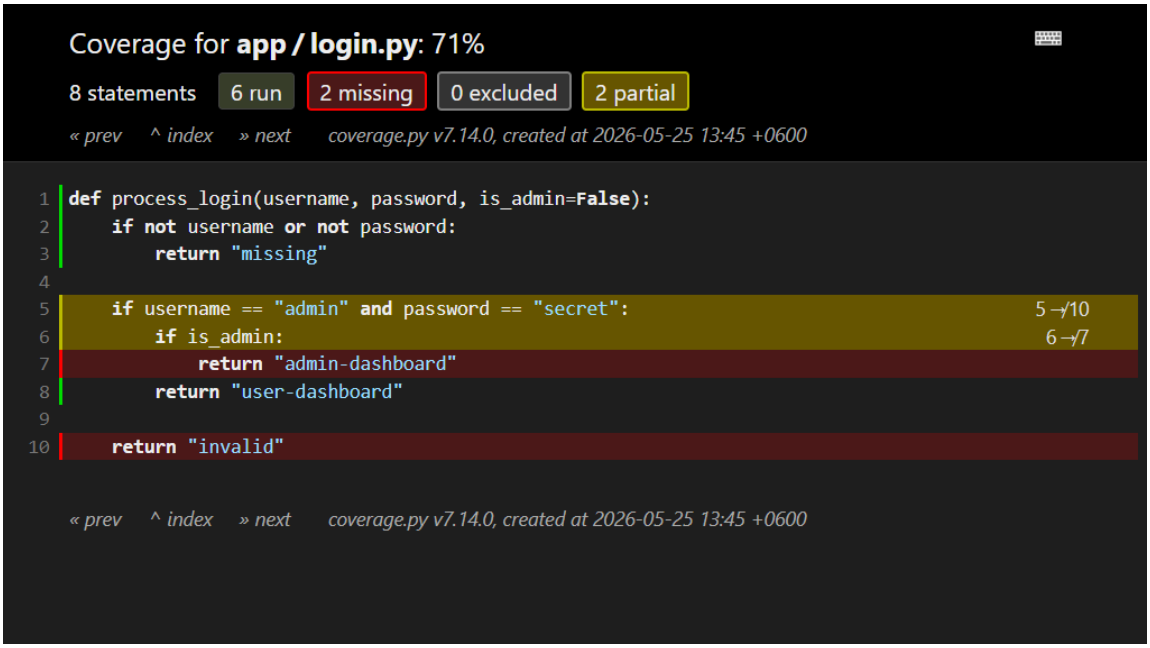


Figure 13: Initial `app/login.py` coverage showing uncovered branches at lines 7 and 10.

5.6 CPG Generation Using Joern

The first attempt used `joern-parse` with the Python language option. This produced an error because the installed Joern version expected a missing `py2cpg.sh` frontend. The available Python frontend was identified as `pysrc2cpg`, so the CPG was generated directly with `pysrc2cpg`.

Listing 7: CPG generation commands

```
joern-parse /home/labuser/maintenance_lab/app --language python --output /home
    /labuser/maintenance_lab/login-cpg.bin
ls -lh /opt/joern/joern-cli | grep cpg
pysrc2cpg --help
pysrc2cpg /home/labuser/maintenance_lab/app -o /home/labuser/maintenance_lab/
    login-cpg.bin
ls -lh login-cpg.bin
```

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab
joern-parse /home/labuser/maintenance_lab/app --language python --output /home/labuser/maintena
nce_lab/login-cpg.bin
Parsing code at: /home/labuser/maintenance_lab/app - language: `python`
[+] Running language frontend
java.lang.RuntimeException: Could not generate CPG with language = python and input = /home/lab
user/maintenance_lab/app
        at io.joern.joerncli.JoernParse$$anon$2.applyOrElse(JoernParse.scala:147)
        at io.joern.joerncli.JoernParse$$anon$2.applyOrElse(JoernParse.scala:144)
        at scala.util.Failure.recover(Try.scala:242)
        at io.joern.joerncli.JoernParse$.generateCpg$$anonfun$2(JoernParse.scala:148)
        at scala.util.Success.flatMap(Try.scala:266)
        at io.joern.joerncli.JoernParse$.generateCpg(JoernParse.scala:140)
        at io.joern.joerncli.JoernParse$.run$$anonfun$2$$anonfun$1(JoernParse.scala:89)
        at scala.util.Success.flatMap(Try.scala:266)
        at io.joern.joerncli.JoernParse$.run$$anonfun$2(JoernParse.scala:88)
        at scala.util.Success.flatMap(Try.scala:266)
        at io.joern.joerncli.JoernParse$.run(JoernParse.scala:87)
        at io.joern.joerncli.JoernParse$.run$$anonfun$1(JoernParse.scala:77)
        at scala.util.Success.flatMap(Try.scala:266)
        at io.joern.joerncli.JoernParse$.run(JoernParse.scala:73)
        at io.joern.joerncli.JoernParse$.main(JoernParse.scala:19)
        at io.joern.joerncli.JoernParse.main(JoernParse.scala)
Caused by: java.lang.AssertionError: assertion failed: CPG generator does not exist at: /opt/jo
ern/joern-cli/py2cpg.sh
        at scala.runtime.Scala3RunTime$.assertFailed(Scala3RunTime.scala:8)
        at io.joern.console.cpgcreation.CpgGenerator.runShellCommand$$anonfun$1(CpgGenerator.sc
ala:30)
        at io.joern.console.cpgcreation.CpgGenerator.runShellCommand$$anonfun$adapted$1(CpgGene
rator.scala:47)
        at scala.util.Try$.apply(Try.scala:217)
        at io.joern.console.cpgcreation.CpgGenerator.runShellCommand(CpgGenerator.scala:47)
        at io.joern.console.cpgcreation.PyCpgGenerator.generate(PyCpgGenerator.scala:19)
        at io.joern.joerncli.JoernParse$.generateCpg$$anonfun$2(JoernParse.scala:143)
        ... 12 more
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ ls -lh /opt/joern/joern-cli | grep cpg
-rwxr-xr-x  1 labuser root  487 May 23 08:44 abap2cpg
-rw-r--r--  1 labuser root  229 May 23 08:44 abap2cpg.bat
-rw-r--r--  1 labuser root  223 May 23 08:44 c2cpg.bat
-rwxr-xr-x  1 labuser root  486 May 23 08:44 c2cpg.sh
-rwxr-xr-x  1 labuser root  497 May 23 08:44 csharpsrc2cpg
-rwxr-xr-x  1 labuser root  238 May 23 08:44 csharpsrc2cpg.bat
-rwxr-xr-x  1 labuser root  441 May 23 08:44 ghidra2cpg
-rw-r--r--  1 labuser root  233 May 23 08:44 ghidra2cpg.bat
-rwxr-xr-x  1 labuser root  489 May 23 08:44 gosrc2cpg
-rw-r--r--  1 labuser root  231 May 23 08:44 gosrc2cpg.bat
-rwxr-xr-x  1 labuser root  492 May 23 08:44 javasrc2cpg
```

Figure 14: Diagnosis of the `joern-parse` Python frontend issue.

```
-rw-r--r--  1 labuser root  231 May 23 08:44 jssrc2cpg.bat
-rwxr-xr-x  1 labuser root  494 May 23 08:44 jssrc2cpg.sh
-rwxr-xr-x  1 labuser root  208 May 23 08:44 kotlin2cpg
-rw-r--r--  1 labuser root  233 May 23 08:44 kotlin2cpg.bat
-rwxr-xr-x  1 labuser root  485 May 23 08:44 php2cpg
-rw-r--r--  1 labuser root  227 May 23 08:44 php2cpg.bat
-rwxr-xr-x  1 labuser root  500 May 23 08:44 pysrc2cpg
-rw-r--r--  1 labuser root  231 May 23 08:44 pysrc2cpg.bat
-rwxr-xr-x  1 labuser root  493 May 23 08:44 rubysrc2cpg
-rw-r--r--  1 labuser root  235 May 23 08:44 rubysrc2cpg.bat
-rwxr-xr-x  1 labuser root  487 May 23 08:44 rust2cpg
-rw-r--r--  1 labuser root  229 May 23 08:44 rust2cpg.bat
-rw-r--r--  1 labuser root  237 May 23 08:44 swiftsrc2cpg.bat
-rwxr-xr-x  1 labuser root  500 May 23 08:44 swiftsrc2cpg.sh
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ find /opt/joern -maxdepth 3 -type f -name "*
cpg.sh" | sort
/opt/joern/joern-cli/c2cpg.sh
/opt/joern/joern-cli/jssrc2cpg.sh
/opt/joern/joern-cli/swiftsrc2cpg.sh
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ pysrc2cpg --help
Usage: pysrc2cpg [options] input-dir

  input-dir               source directory
  -o, --output <value>    output filename
  --exclude <file1>       files or folders to exclude during CPG generation (paths relative to
 <input-dir> or absolute paths)
  --exclude-regex <value>  a regex specifying files to exclude during CPG generation (paths rel
ative to <input-dir> are matched)
  --enable-early-schema-checking
                          enables early schema validation during AST creation (disabled by def
ault)
  --enable-file-content   add the raw source code to the content field of FILE nodes to allow
for method source retrieval via offset fields (disabled by default)
  --help                  display this help message
  --venvDirs <value>      Virtual environment directories. If not absolute they are interprete
d relative to input-dir.
  --ignoreVenvDir <value>  Specifies whether venv-dir is ignored. Default to true.
  --ignore-paths <value>  Ignores the specified path from analysis. If not absolute it is inte
rpreted relative to input-dir.
  --ignore-dir-names <value>
                          Excludes all files where the relative path from input-dir contains a
t least one of names specified here.
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

pysrc2cpg /home/labuser/maintenance_lab/app -o /home/labuser/maintenance_lab/login-cpg.bin

ls -lh login-cpg.bin
-rw-r--r-- 1 labuser labuser 53K May 25 14:16 login-cpg.bin
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 15: Successful CPG generation using `pysrc2cpg`.

5.7 Loading and Querying the CPG in Joern

The generated CPG was loaded into the Joern shell. Method names, file names, control structures, calls, literals, identifiers, and AST nodes were queried.

Listing 8: Joern CPG query commands

```
importCpg("/home/labuser/maintenance_lab/login-cpg.bin")
cpg.method.name.l
cpg.file.name.l
cpg.controlStructure.code.l
```

```
cpg.controlStructure.map(c => (c.lineNumber, c.code)).l
cpg.call.code.l
cpg.literal.code.l
cpg.identifier.name.l
cpg.method("process_login").ast.code.l
```



Figure 16: CPG loaded into Joern and method names displayed.

```
joern> cpg.method("process_login").code.l
val res3: List[String] = List("<empty>")

joern> cpg.controlStructure.code.l
val res4: List[String] = List("if ... : ...", "if ... : ...", "if ... : ...")

joern> cpg.controlStructure.map(c => (c.lineNumber, c.code)).l
val res5: List[(Option[Int], String)] = List(
  (Some(value = 2), "if ... : ..."),
  (Some(value = 6), "if ... : ..."),
  (Some(value = 5), "if ... : ...")
)

joern> cpg.call.code.l
val res6: List[String] = List(
  "not username",
  "not password",
  "not username or not password",
  "username == \"admin\"",
  "password == \"secret\"",
  "username == \"admin\" and password == \"secret\"",
  "process_login = def process_login(...)"
)

joern>
```

Figure 17: Joern CPG queries showing control structures and branch-related call expressions.

```
joern> cpg.literal.code.l
val res7: List[String] = List(
  "\"missing\"",
  "\"admin\"",
  "\"secret\"",
  "\"admin-dashboard\"",
  "\"user-dashboard\"",
  "\"invalid\""
)

joern> cpg.identifier.name.l
val res8: List[String] = List("process_login", "username", "password", "username", "password", "is_admin
")

joern> cpg.method("process_login").ast.code.l
val res9: List[String] = List(
  "<empty>",
  "",
  "if ... : ...",
  "not username or not password",
  "not username",
  "username",
  "not password",
  "password",
  "return \"missing",
  "return \"missing",
  "\"missing\"",
  "if ... : ...",
  "username == \"admin\" and password == \"secret\"",
  "username == \"admin\"",
  "username",
  "\"admin\"",
  "password == \"secret\"",
  "password",
  "\"secret\"",
  """if ... : ...
```

Figure 18: Joern queries showing literals, identifiers, and AST nodes.



Figure 19: Continuation of AST output for the `process_login` function.

5.8 Exporting the CPG

The CPG was exported in Neo4j-compatible CSV format and DOT format. The Neo4j CSV export was used for graph database import, while the DOT export was converted into a PNG graph using Graphviz.

Listing 9: CPG export commands

```
joern-export /home/labuser/maintenance_lab/login-cpg.bin \
  --out /home/labuser/maintenance_lab/cpg-export-neo4j \
  --repr all \
  --format neo4jcsv

joern-export /home/labuser/maintenance_lab/login-cpg.bin \
  --out /home/labuser/maintenance_lab/cpg-export-dot \
  --repr all \
  --format dot

dot -Tpng cpg-export-dot/export.dot -o cpg_graph.png
```

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

rm -rf cpg-export-neo4j

joern-export /home/labuser/maintenance_lab/login-cpg.bin \
  --out /home/labuser/maintenance_lab/cpg-export-neo4j \
  --repr all \
  --format neo4jcsv

ls -lh cpg-export-neo4j
CPG does not have dataflow overlay. Calculating.
exported 65 nodes, 91 edges into /home/labuser/maintenance_lab/cpg-export-neo4j
Instructions on how to import the exported files into neo4j:
Prerequisite: ensure you have neo4j community server running (enterprise and desktop may work too)
e.g. download from https://neo4j.com/download-center/#community and start via `bin/neo4j console`

Then, in a new terminal:
```
cd <neo4j_root>

if you have a fresh instance, you must first change the initial password
bin/cypher-shell -u neo4j -p neo4j
exit the cypher shell

copy the data files to the `import` directory, where neo4j will find them
cp /home/labuser/maintenance_lab/cpg-export-neo4j/*_data.csv import

find /home/labuser/maintenance_lab/cpg-export-neo4j -name 'nodes_*_cypher.csv' -exec bin/cypher-shell -u
 neo4j -p <password> --file {} \;
find /home/labuser/maintenance_lab/cpg-export-neo4j -name 'edges_*_cypher.csv' -exec bin/cypher-shell -u
 neo4j -p <password> --file {} \;
```

total 300K
-rw-r--r-- 1 labuser labuser  164 May 25 14:36 edges_ARGUMENT_cypher.csv
-rw-r--r-- 1 labuser labuser  549 May 25 14:36 edges_ARGUMENT_data.csv
-rw-r--r-- 1 labuser labuser   25 May 25 14:36 edges_ARGUMENT_header.csv
-rw-r--r-- 1 labuser labuser  154 May 25 14:36 edges_AST_cypher.csv
-rw-r--r-- 1 labuser labuser 1.6K May 25 14:36 edges_AST_data.csv
-rw-r--r-- 1 labuser labuser   25 May 25 14:36 edges_AST_header.csv
-rw-r--r-- 1 labuser labuser  158 May 25 14:36 edges_BINDS_cypher.csv
-rw-r--r-- 1 labuser labuser   96 May 25 14:36 edges_BINDS_data.csv
-rw-r--r-- 1 labuser labuser   25 May 25 14:36 edges_BINDS_header.csv
-rw-r--r-- 1 labuser labuser  166 May 25 14:36 edges_CONDITION_cypher.csv
-rw-r--r-- 1 labuser labuser  105 May 25 14:36 edges_CONDITION_data.csv
-rw-r--r-- 1 labuser labuser   25 May 25 14:36 edges_CONDITION_header.csv
-rw-r--r-- 1 labuser labuser  189 May 25 14:36 edges_REACHING_DEF_cypher.csv
-rw-r--r-- 1 labuser labuser  123 May 25 14:36 edges_REACHING_DEF_data.csv
-rw-r--r-- 1 labuser labuser   41 May 25 14:36 edges_REACHING_DEF_header.csv
```

Figure 20: CPG exported to Neo4j-compatible CSV format.

```
-rw-r--r-- 1 labuser labuser  581 May 25 14:36 nodes_CALL_cypher.csv
-rw-r--r-- 1 labuser labuser  901 May 25 14:36 nodes_CALL_data.csv
-rw-r--r-- 1 labuser labuser  297 May 25 14:36 nodes_CALL_header.csv
-rw-r--r-- 1 labuser labuser  437 May 25 14:36 nodes_CONTROL_STRUCTURE_cypher.csv
-rw-r--r-- 1 labuser labuser  221 May 25 14:36 nodes_CONTROL_STRUCTURE_data.csv
-rw-r--r-- 1 labuser labuser  196 May 25 14:36 nodes_CONTROL_STRUCTURE_header.csv
-rw-r--r-- 1 labuser labuser  268 May 25 14:36 nodes_FILE_cypher.csv
-rw-r--r-- 1 labuser labuser  105 May 25 14:36 nodes_FILE_data.csv
-rw-r--r-- 1 labuser labuser  110 May 25 14:36 nodes_FILE_header.csv
-rw-r--r-- 1 labuser labuser  492 May 25 14:36 nodes_IDENTIFIER_cypher.csv
-rw-r--r-- 1 labuser labuser  405 May 25 14:36 nodes_IDENTIFIER_data.csv
-rw-r--r-- 1 labuser labuser  219 May 25 14:36 nodes_IDENTIFIER_header.csv
-rw-r--r-- 1 labuser labuser  484 May 25 14:36 nodes_LITERAL_cypher.csv
-rw-r--r-- 1 labuser labuser  451 May 25 14:36 nodes_LITERAL_data.csv
-rw-r--r-- 1 labuser labuser  216 May 25 14:36 nodes_LITERAL_header.csv
-rw-r--r-- 1 labuser labuser  431 May 25 14:36 nodes_LOCAL_cypher.csv
-rw-r--r-- 1 labuser labuser   70 May 25 14:36 nodes_LOCAL_data.csv
-rw-r--r-- 1 labuser labuser  207 May 25 14:36 nodes_LOCAL_header.csv
-rw-r--r-- 1 labuser labuser  215 May 25 14:36 nodes_META_DATA_cypher.csv
-rw-r--r-- 1 labuser labuser   90 May 25 14:36 nodes_META_DATA_data.csv
-rw-r--r-- 1 labuser labuser   78 May 25 14:36 nodes_META_DATA_header.csv
-rw-r--r-- 1 labuser labuser  545 May 25 14:36 nodes_METHOD_PARAMETER_IN_cypher.csv
-rw-r--r-- 1 labuser labuser  273 May 25 14:36 nodes_METHOD_PARAMETER_IN_data.csv
-rw-r--r-- 1 labuser labuser  247 May 25 14:36 nodes_METHOD_PARAMETER_IN_header.csv
-rw-r--r-- 1 labuser labuser  504 May 25 14:36 nodes_METHOD_REF_cypher.csv
-rw-r--r-- 1 labuser labuser  130 May 25 14:36 nodes_METHOD_REF_data.csv
-rw-r--r-- 1 labuser labuser  231 May 25 14:36 nodes_METHOD_REF_header.csv
-rw-r--r-- 1 labuser labuser  425 May 25 14:36 nodes_METHOD_RETURN_cypher.csv
-rw-r--r-- 1 labuser labuser  181 May 25 14:36 nodes_METHOD_RETURN_data.csv
-rw-r--r-- 1 labuser labuser  186 May 25 14:36 nodes_METHOD_RETURN_header.csv
-rw-r--r-- 1 labuser labuser  559 May 25 14:36 nodes_METHOD_cypher.csv
-rw-r--r-- 1 labuser labuser  366 May 25 14:36 nodes_METHOD_data.csv
-rw-r--r-- 1 labuser labuser  299 May 25 14:36 nodes_METHOD_header.csv
-rw-r--r-- 1 labuser labuser  251 May 25 14:36 nodes_MODIFIER_cypher.csv
-rw-r--r-- 1 labuser labuser  223 May 25 14:36 nodes_MODIFIER_data.csv
-rw-r--r-- 1 labuser labuser   99 May 25 14:36 nodes_MODIFIER_header.csv
-rw-r--r-- 1 labuser labuser  296 May 25 14:36 nodes_NAMESPACE_BLOCK_cypher.csv
-rw-r--r-- 1 labuser labuser  233 May 25 14:36 nodes_NAMESPACE_BLOCK_data.csv
-rw-r--r-- 1 labuser labuser  123 May 25 14:36 nodes_NAMESPACE_BLOCK_header.csv
-rw-r--r-- 1 labuser labuser  353 May 25 14:36 nodes_RETURN_cypher.csv
-rw-r--r-- 1 labuser labuser  253 May 25 14:36 nodes_RETURN_data.csv
-rw-r--r-- 1 labuser labuser  142 May 25 14:36 nodes_RETURN_header.csv
-rw-r--r-- 1 labuser labuser  548 May 25 14:36 nodes_TYPE_DECL_cypher.csv
-rw-r--r-- 1 labuser labuser  454 May 25 14:36 nodes_TYPE_DECL_data.csv
-rw-r--r-- 1 labuser labuser  294 May 25 14:36 nodes_TYPE_DECL_header.csv
-rw-r--r-- 1 labuser labuser  158 May 25 14:36 nodes_TYPE_cypher.csv
-rw-r--r-- 1 labuser labuser  262 May 25 14:36 nodes_TYPE_data.csv
-rw-r--r-- 1 labuser labuser   68 May 25 14:36 nodes_TYPE_header.csv
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 21: Generated Neo4j CSV files containing CPG node, edge, and Cypher import files.

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

echo "CONTROL STRUCTURE NODES:"
cat cpg-export-neo4j/nodes_CONTROL_STRUCTURE_data.csv

echo ""
echo "LITERAL NODES:"
cat cpg-export-neo4j/nodes_LITERAL_data.csv

echo ""
echo "CONDITION EDGES:"
cat cpg-export-neo4j/edges_CONDITION_data.csv
CONTROL STRUCTURE NODES:
47244640256,CONTROL_STRUCTURE,-1,,,if ... : ...,5,IF,2,59,122,1,<empty>
47244640257,CONTROL_STRUCTURE,-1,,,if ... : ...,9,IF,6,179,237,1,<empty>
47244640258,CONTROL_STRUCTURE,-1,,,if ... : ...,5,IF,5,122,266,2,<empty>

LITERAL NODES:
90194313216,LITERAL,1,,,"""missing""",16,__builtin.str,3,107,115,1,,ANY
90194313217,LITERAL,2,,,"""admin""",20,__builtin.str,5,137,143,2,,ANY
90194313218,LITERAL,2,,,"""secret""",44,__builtin.str,5,161,168,2,,ANY
90194313219,LITERAL,1,,,"""admin-dashboard""",20,__builtin.str,7,211,227,1,,ANY
90194313220,LITERAL,1,,,"""user-dashboard""",16,__builtin.str,8,244,259,1,,ANY
90194313221,LITERAL,1,,,"""invalid""",12,__builtin.str,10,273,281,1,,ANY

CONDITION EDGES:
47244640256,30064771074,CONDITION
47244640257,68719476741,CONDITION
47244640258,30064771077,CONDITION
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 22: Inspection of exported CPG CSV content including control-structure nodes, literal nodes, and condition edges.

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

rm -rf cpg-export-dot

joern-export /home/labuser/maintenance_lab/login-cpg.bin \
  --out /home/labuser/maintenance_lab/cpg-export-dot \
  --repr all \
  --format dot

find cpg-export-dot -type f | head -20
exported 65 nodes, 91 edges into /home/labuser/maintenance_lab/cpg-export-dot
cpg-export-dot/export.dot
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 23: CPG exported as DOT graph file.

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

dot -Tpng cpg-export-dot/export.dot -o cpg_graph.png

ls -lh cpg_graph.png
-rw-r--r-- 1 labuser labuser 539K May 25 14:44 cpg_graph.png
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 24: DOT graph converted into PNG using Graphviz.

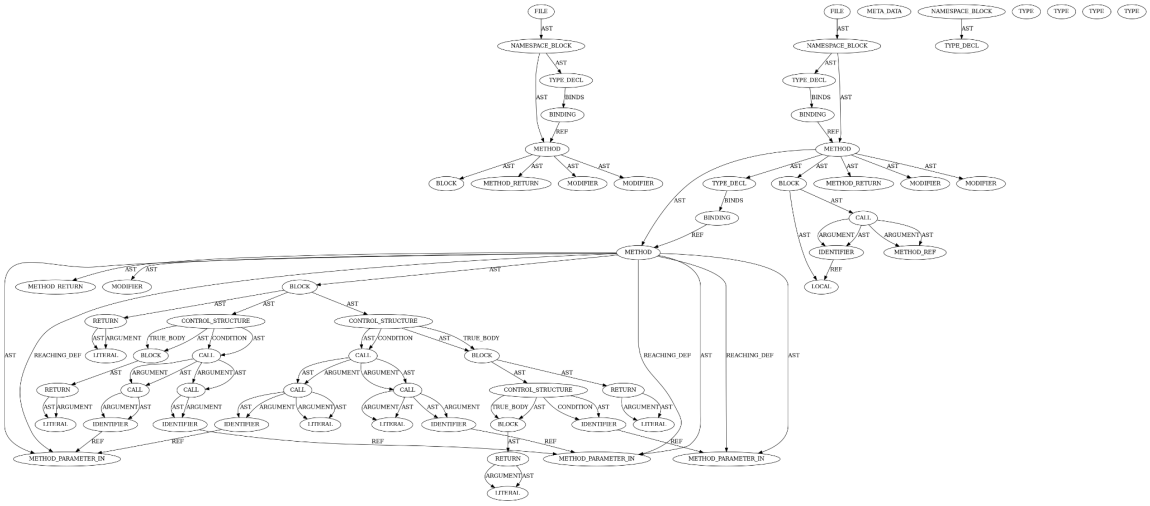


Figure 25: Visual CPG graph generated from DOT file.

5.9 Branch-Targeted Test Improvement

The initial coverage report showed missing branches at lines 7 and 10. Therefore, two targeted tests were added: one for the admin-dashboard branch and one for the invalid-login branch.

Listing 10: Branch-targeted tests added to tests_test_login.py

```
def test_admin_login():
    assert process_login("admin", "secret", is_admin=True) == "admin-dashboard
    "


def test_invalid_login():
    assert process_login("john", "wrong") == "invalid"
```

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

cat >> tests/test_login.py <<'PY'

def test_admin_login():
    assert process_login("admin", "secret", is_admin=True) == "admin-dashboard"

def test_invalid_login():
    assert process_login("john", "wrong") == "invalid"
PY

cat tests/test_login.py
from app.login import process_login

def test_missing_username():
    assert process_login("", "secret") == "missing"

def test_valid_user_login():
    assert process_login("admin", "secret") == "user-dashboard"

def test_admin_login():
    assert process_login("admin", "secret", is_admin=True) == "admin-dashboard"

def test_invalid_login():
    assert process_login("john", "wrong") == "invalid"
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

coverage run --branch -m pytest -q
coverage report -m
....                                                                     [100%]
4 passed in 0.09s
Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
app/__init__.py           0      0      0      0   100%
app/login.py              8      0      6      0   100%
tests/test_login.py       9      0      0      0   100%
-----------------------------------------------------------------
TOTAL                    17      0      6      0   100%
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$
```

Figure 26: Final pytest and branch coverage report after adding targeted tests.

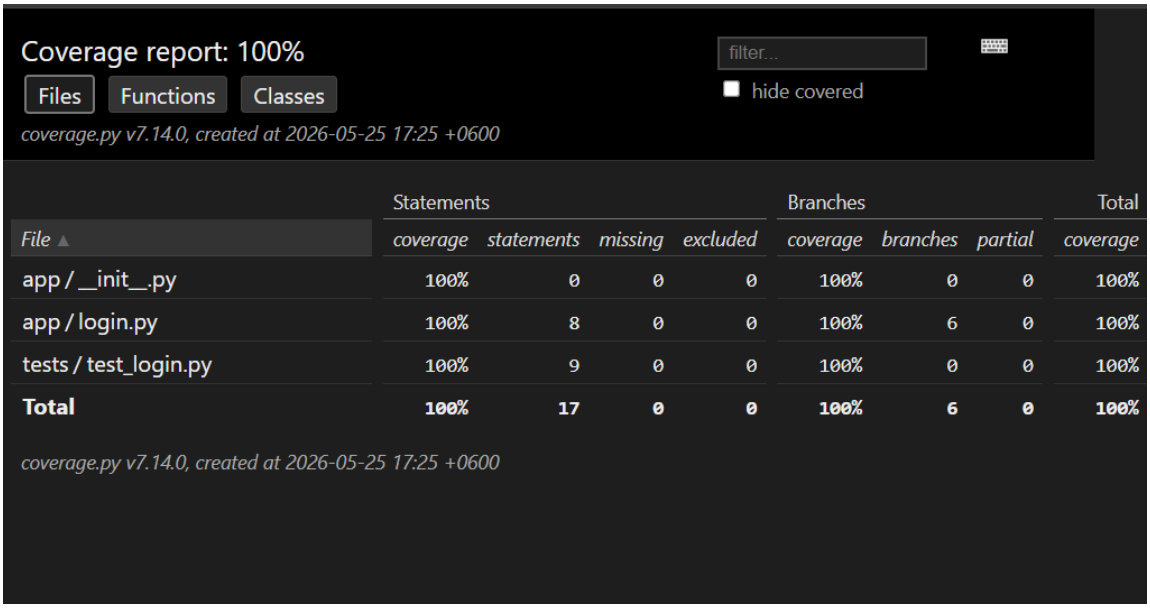


Figure 27: Final HTML coverage summary showing 100% total coverage.



Figure 28: Final `app/login.py` coverage showing all branches covered.

```
(.venv) labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

cat > lab_result_summary.txt <<'TXT'
Maintenance Lab Result Summary

Project:
CPG-based branch-aware testing using Joern, coverage.py, and pytest.

Initial Test Result:
- Number of tests: 2
- Result: 2 passed
- Total coverage: 79%
- app/login.py coverage: 71%
- Missing lines: 7, 10
- Missing branches:
  1. return "admin-dashboard"
  2. return "invalid"

CPG Analysis:
- Joern generated CPG file: login-cpg.bin
- CPG size: 53 KB
- CPG graph: 104 nodes inside Joern
- Neo4j CSV export: 65 nodes, 91 edges
- DOT graph export: 65 nodes, 91 edges
- Important CPG node types:
  FILE, METHOD, BLOCK, CONTROL_STRUCTURE, CALL, IDENTIFIER, LITERAL, RETURN

Branch-Targeted Tests Added:
1. test_admin_login()
2. test_invalid_login()

Final Test Result:
- Number of tests: 4
- Result: 4 passed
- Total coverage: 100%
- app/login.py coverage: 100%
- Missing lines: 0
- Partial branches: 0

Conclusion:
Using CPG-based structural analysis, the uncovered branches were identified and targeted test ca
ses were added. This improved branch coverage from 79% to 100%.
TXT

cat lab_result_summary.txt
```

Figure 29: Creation of final lab result summary file.

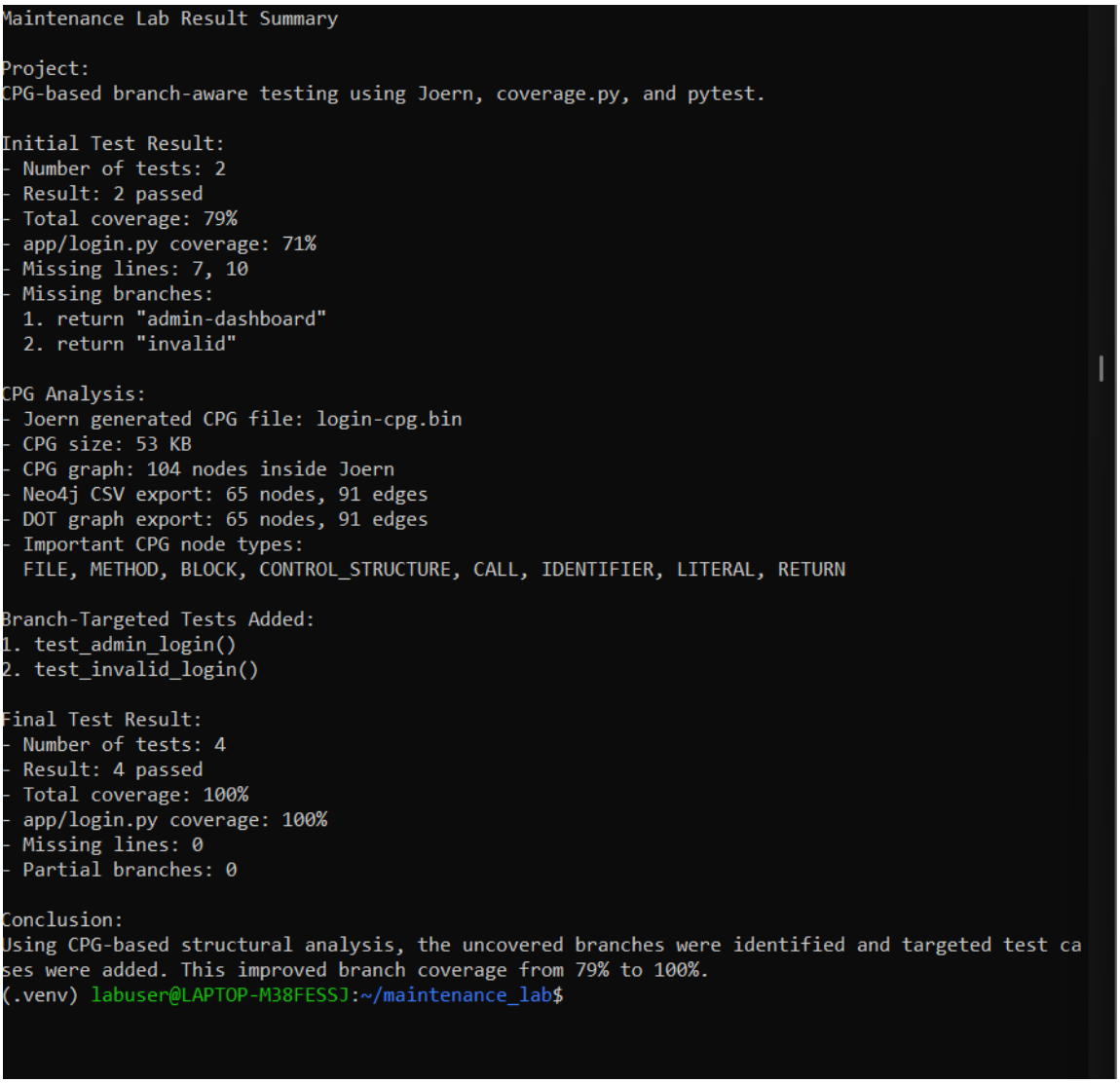


Figure 30: Final lab result summary displayed in terminal.

5.10 Neo4j Installation and CPG Import

Neo4j Community Edition was downloaded and prepared inside Ubuntu. After starting the Neo4j server, the Neo4j Browser was accessed from the browser and connected using the configured credentials. The exported CPG CSV files were copied into the Neo4j import folder. Node files were imported first, followed by relationship files.

Listing 11: Neo4j download, preparation, and CPG import commands

```
wget -O neo4j-community-5.26.0-unix.tar.gz https://dist.neo4j.org/neo4j-
    community-5.26.0-unix.tar.gz
tar -xzf neo4j-community-5.26.0-unix.tar.gz
mv neo4j-community-5.26.0 neo4j
./neo4j/bin/neo4j-admin dbms set-initial-password labpass123
./neo4j/bin/neo4j console
```

```
cp cpg-export-neo4j/*_data.csv neo4j/import/
find cpg-export-neo4j -name 'nodes_*_cypher.csv' -exec ./neo4j/bin/cypher-
    shell -a bolt://localhost:7687 -u neo4j -p labpass123 --file {} \;
find cpg-export-neo4j -name 'edges_*_cypher.csv' -exec ./neo4j/bin/cypher-
    shell -a bolt://localhost:7687 -u neo4j -p labpass123 --file {} \;
```

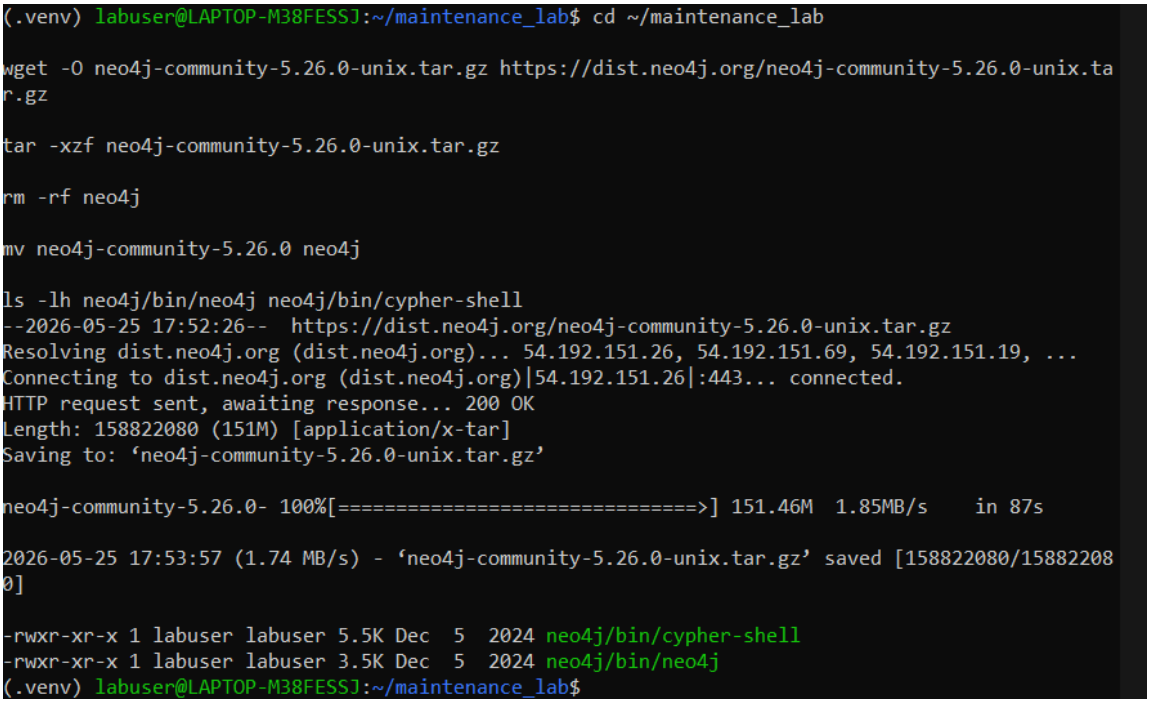


Figure 31: Neo4j downloaded and prepared in Ubuntu.

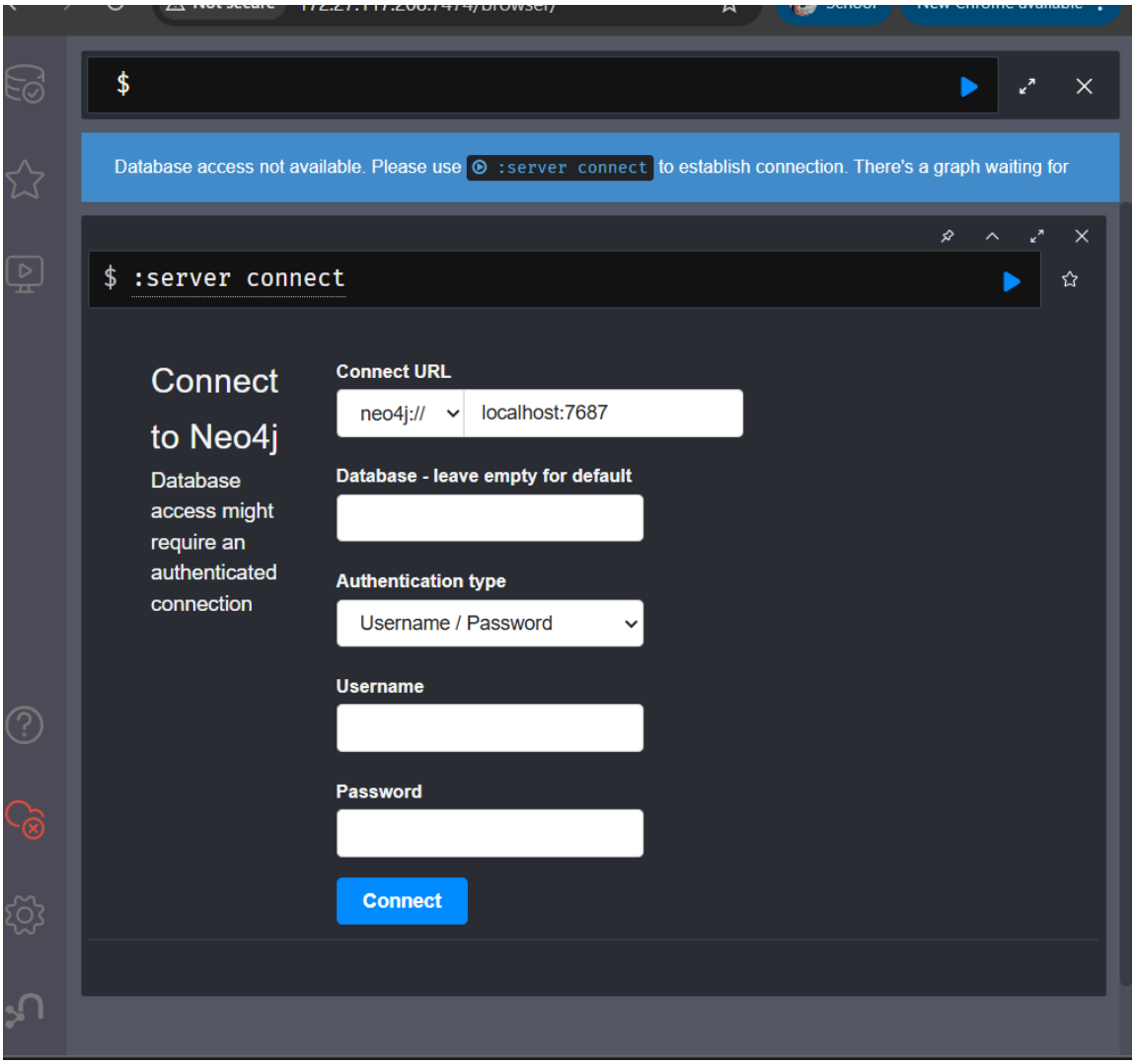


Figure 32: Neo4j Browser connection form.

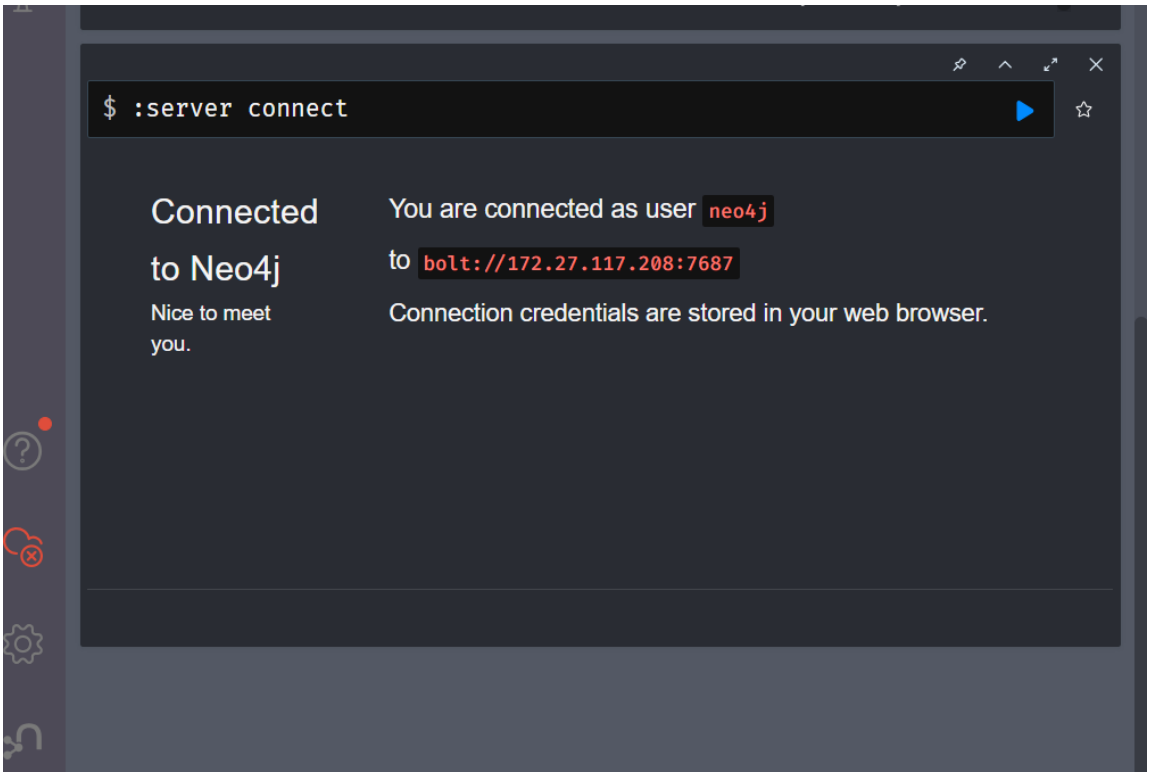


Figure 33: Neo4j Browser connected successfully.

```
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

cp cpg-export-neo4j/*_data.csv neo4j/import/

ls neo4j/import | head
edges_ARGUMENT_data.csv
edges_AST_data.csv
edges_BINDS_data.csv
edges_CONDITION_data.csv
edges_REACHING_DEF_data.csv
edges_REF_data.csv
edges_TRUE_BODY_data.csv
nodes_BINDING_data.csv
nodes_BLOCK_data.csv
nodes_CALL_data.csv
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ find cpg-export-neo4j -name 'nodes_*_cypher.csv' -exec ./neo4j/bin/cypher-shell -a bolt://localhost:7687 -u neo4j
 -p labpass123 --file {} \;
0 rows
ready to start consuming query after 516 ms, results consumed after another 0 ms
Added 3 nodes, Set 18 properties, Added 3 labels
0 rows
ready to start consuming query after 117 ms, results consumed after another 0 ms
Added 7 nodes, Set 84 properties, Added 7 labels
0 rows
ready to start consuming query after 91 ms, results consumed after another 0 ms
Added 1 nodes, Set 8 properties, Added 1 labels
0 rows
ready to start consuming query after 88 ms, results consumed after another 0 ms
Added 6 nodes, Set 51 properties, Added 6 labels
0 rows
ready to start consuming query after 37 ms, results consumed after another 0 ms
Added 4 nodes, Set 16 properties, Added 4 labels
0 rows
ready to start consuming query after 96 ms, results consumed after another 0 ms
Added 4 nodes, Set 59 properties, Added 4 labels
0 rows
ready to start consuming query after 80 ms, results consumed after another 0 ms
Added 2 nodes, Set 10 properties, Added 2 labels
0 rows
ready to start consuming query after 51 ms, results consumed after another 0 ms
Added 3 nodes, Set 30 properties, Added 3 labels
```

Figure 34: Importing CPG node CSV files into Neo4j.

```
ready to start consuming query after 68 ms, results consumed after another 0 ms
Added 3 nodes, Set 48 properties, Added 3 labels
0 rows
ready to start consuming query after 28 ms, results consumed after another 0 ms
Added 3 nodes, Set 6 properties, Added 3 labels
0 rows
ready to start consuming query after 44 ms, results consumed after another 0 ms
Added 5 nodes, Set 20 properties, Added 5 labels
0 rows
ready to start consuming query after 45 ms, results consumed after another 0 ms
Added 1 nodes, Set 5 properties, Added 1 labels
0 rows
ready to start consuming query after 43 ms, results consumed after another 0 ms
Added 1 nodes, Set 10 properties, Added 1 labels
0 rows
ready to start consuming query after 38 ms, results consumed after another 0 ms
Added 3 nodes, Set 27 properties, Added 3 labels
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

find cpg-export-neo4j -name 'edges_*_cypher.csv' -exec ./neo4j/bin/cypher-shell -a bolt://localhost:7687 -u neo4j -p labpass123 --file {} \;
0 rows
ready to start consuming query after 153 ms, results consumed after another 0 ms
Created 3 relationships
0 rows
ready to start consuming query after 56 ms, results consumed after another 0 ms
Created 3 relationships
0 rows
ready to start consuming query after 85 ms, results consumed after another 0 ms
Created 3 relationships
0 rows
ready to start consuming query after 48 ms, results consumed after another 0 ms
Created 3 relationships
0 rows
ready to start consuming query after 121 ms, results consumed after another 0 ms
Created 16 relationships
0 rows
ready to start consuming query after 117 ms, results consumed after another 0 ms
Created 54 relationships
0 rows
ready to start consuming query after 59 ms, results consumed after another 0 ms
Created 9 relationships
```

Figure 35: Importing CPG edge CSV files into Neo4j.

```
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ cd ~/maintenance_lab

./neo4j/bin/cypher-shell -a bolt://localhost:7687 -u neo4j -p labpass123 "MATCH (n) RETURN labels(n) AS labels, count(n) AS count ORDER BY count DESC;"
+---------------------------------+
| labels                  | count |
+---------------------------------+
| ["CALL"]                | 7     |
| ["BLOCK"]               | 6     |
| ["IDENTIFIER"]          | 6     |
| ["LITERAL"]             | 6     |
| ["MODIFIER"]            | 5     |
| ["TYPE"]                | 4     |
| ["TYPE_DECL"]           | 4     |
| ["RETURN"]              | 4     |
| ["NAMESPACE_BLOCK"]     | 3     |
| ["CONTROL_STRUCTURE"]   | 3     |
| ["METHOD_PARAMETER_IN"] | 3     |
| ["METHOD"]              | 3     |
| ["BINDING"]             | 3     |
| ["METHOD_RETURN"]       | 3     |
| ["FILE"]                | 2     |
| ["LOCAL"]               | 1     |
| ["META_DATA"]           | 1     |
| ["METHOD_REF"]          | 1     |
+---------------------------------+

18 rows
ready to start consuming query after 71 ms, results consumed after another 7 ms
labuser@LAPTOP-M38FESSJ:~/maintenance_lab$ ./neo4j/bin/cypher-shell -a bolt://localhost:7687 -u neo4j -p labpass123 "MATCH ()-[r]->() RETURN type(r) AS rela
tionship, count(r) AS count ORDER BY count DESC;"
+------------------------+
| relationship | count |
+------------------------+
| "AST"        | 54    |
| "ARGUMENT"   | 16    |
| "REF"        | 9     |
| "TRUE_BODY"  | 3     |
| "CONDITION"  | 3     |
| "REACHING_DEF" | 3     |
| "BINDS"      | 3     |
+------------------------+
```

Figure 36: Verification of imported CPG node labels and relationship types in Neo4j.

5.11 Neo4j Visualization

The CPG was visualized in Neo4j Browser using Cypher queries. The full graph, AST subgraph, control-structure-condition relationships, and literal return values were inspected.

Listing 12: Cypher queries used for Neo4j visualization

```
MATCH p=()-->() RETURN p LIMIT 100;

MATCH p=(m:METHOD {NAME: "process_login"})-[:AST*1..6]->(n)
RETURN p
```

```
LIMIT 100;

MATCH p=(cs:CONTROL_STRUCTURE)-[:CONDITION]->(cond)
RETURN p;

MATCH (n:LITERAL)
RETURN n.CODE AS Literal, n.LINE_NUMBER AS Line
ORDER BY Line;
```



Figure 37: Full CPG graph visualization in Neo4j Browser.

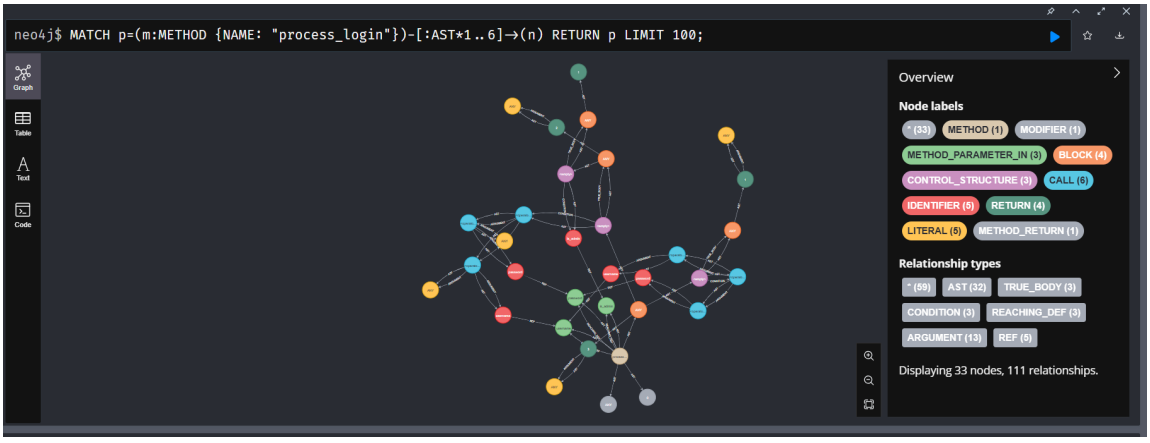


Figure 38: AST visualization of `process_login` method in Neo4j.

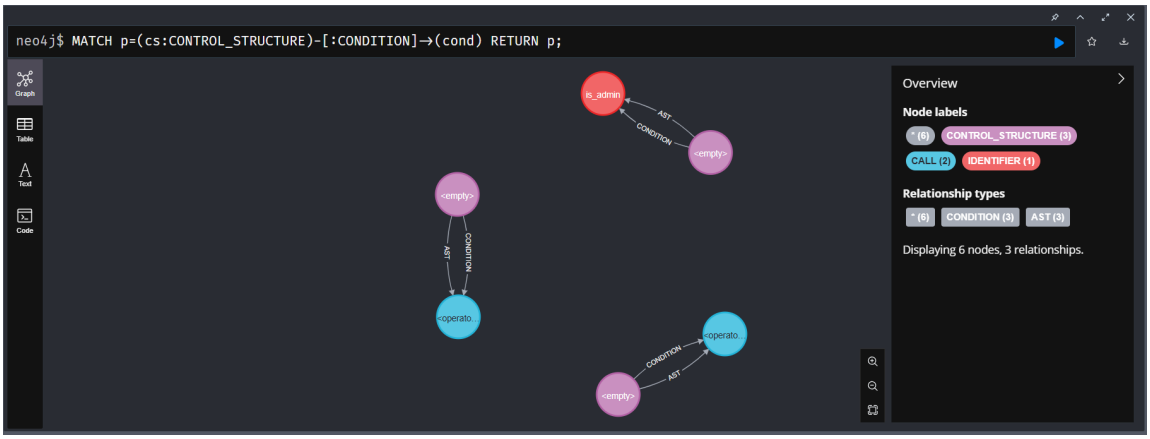


Figure 39: Control-structure and condition visualization in Neo4j.

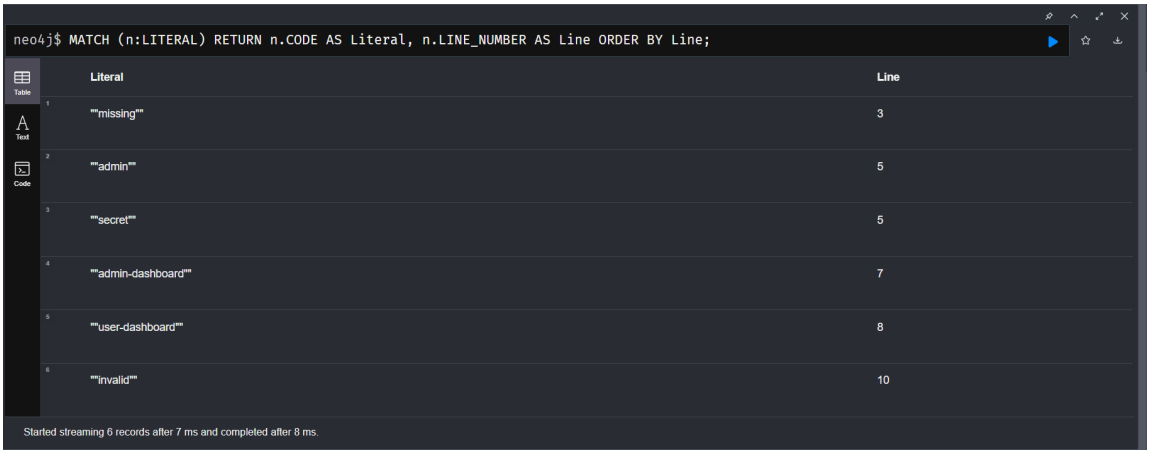


Figure 40: Literal and return values extracted from the CPG in Neo4j.

6 Results

6.1 Coverage Improvement

Table 2: Coverage before and after branch-targeted test generation

Stage	Tests	app/login.py Coverage	Total Coverage	Missing Lines
Initial testing	2	71%	79%	7, 10
After branch-targeted tests	4	100%	100%	None

6.2 CPG and Neo4j Import Summary

Table 3: CPG generation and import summary

Item	Result
Generated CPG file	`login-cpg.bin`
CPG file size	53 KB
Joern graph size	104 nodes inside Joern
Neo4j CSV export	65 nodes and 91 edges
DOT export	65 nodes and 91 edges
Important node types	`FILE`, `METHOD`, `BLOCK`, `CONTROL_STRUCTURE`, `CALL`, `IDENTIFIER`, `LITERAL`, `RETURN`
Important relationships	`AST`, `ARGUMENT`, `REF`, `TRUE_BODY`, `CONDITION`, `REACHING_DEF`, `BINDS`

7 Discussion

The experiment demonstrated the usefulness of CPG-based structural analysis in software maintenance testing. The initial tests were functionally correct, but they did not cover all branch outcomes. The coverage report showed that `return "admin-dashboard"` and `return "invalid"` were not executed. Joern's CPG queries confirmed the existence of the corresponding control structures, literals, identifiers, and return values.

The CPG export and Neo4j visualization provided a graph-oriented view of the program. Instead of only reading source code linearly, the graph made it possible to inspect relationships among methods, calls, literals, identifiers, control structures, and conditions. This connects directly to the motivation of program structure-aware testing: source code contains structural signals that can guide better testing decisions.

The most important practical outcome was the improvement in coverage. After identifying uncovered branches, two additional branch-targeted tests were added. This increased `app/login.py` coverage from 71% to 100% and total coverage from 79% to 100%. Therefore, the lab successfully reproduced the core idea of CPG-assisted branch-aware testing on a small project.

8 Limitations

This lab used a small Python function instead of a large real-world repository. The workflow reproduced the practical idea of CPG-based branch-aware testing, but it did not train a graph neural network or language model. In the GLMTest paper, CPG embeddings are combined with language-model prompts for targeted test generation; that full pipeline requires large datasets, GPU resources, and model training infrastructure. Therefore, this lab focused on the practical

maintenance workflow: CPG generation, graph visualization, branch identification, and test improvement.

9 Conclusion

This lab successfully implemented a complete Code Property Graph workflow for software maintenance testing. Joern was installed and used to generate a CPG from Python source code. The CPG was queried inside Joern, exported to Neo4j-compatible CSV format, visualized as a DOT graph, and imported into Neo4j for graph exploration. Initial branch coverage revealed two uncovered branches. After analyzing the program structure and coverage report, two branch-targeted tests were added. The final result showed 100% statement and branch coverage. This confirms that CPG-based structural analysis can support more systematic and effective software testing during maintenance.